

사내 연차 경매 시스템

Annual Leave Auction System — 요구분석 및 설계 결과

팀 타임소프트콘 · 김기철 · 오지석

공학대 2학년 1학기 소프트웨어공학 · 프로젝트 1 (13주차)

고연차의 *버려지는 연차*



저연차의 *부족한 연차* 를

B2E Escrow & Dividend 아키텍처로 잇는,
사내 금융 수준 정합성을 갖춘 연차 거래 플랫폼

① 프로젝트 개요

해결하려는 문제 · 사용자 · 핵심 기능

① 문제 정의 — 사내 "연차 양극화"

집단	상황	손해
고연차 직원	업무 과다로 연차를 다 못 씀	연말에 미사용 연차 소멸 (0원)
저연차 직원	법정 연차가 적음	휴가 부족 호소

- 양쪽 모두 손해인 현상이 매년 반복 → 사내 복지 비효율
- 단순 아이디어: "직원끼리 연차를 사고팔자" (P2P)
 - ✗
 - 폐기: 근로기준법 제60·61조 — 휴식권의 매매는 위법
 - → 개인 간 직접 거래(P2P)·현금 결제는 설계에서 영구 배제

핵심 도전 과제: **합법적이면서도 회사 재무 리스크 0%**인 매칭 구조를 어떻게 만들 것인가?

① 해결책 — B2E Escrow & Dividend



수익적립금 = 구매자가 입찰로 낸 포인트를, 회사가 마음대로 쓰지 못하게 따로 모아두는 공동의 돈. 연말에 이 모인 돈만큼만 판매자에게 나눠줍니다.

중고거래 안전결제처럼 *모아뒀다 정산* — 영문 아키텍처명에선 *Escrow*.

회사가 중개하고(B2E), 대가는 복지 포인트, 정산은 연말 일괄 배당.

전환 축	초기 아이디어	최종 설계
거래 주체	직원 ↔ 직원 (P2P)	회사가 중개 (B2E)
재화	현금(KRW)	사내 복지 포인트
정산 시점	즉시 지급	연말 일괄 배당
매물 단위	날짜 종속 연차	"연차 1일권" (개수, 날짜 비종속)

- 구매자가 낸 포인트는 회사 예산이 아니라 **수익적립금**으로 차곡차곡 쌓임
- 연말에 판매자(기여자)에게 **지분(Stake = 내가 기여한 연차의 비율)**대로 복지카드 한도로 배당
- 회사는 **예산 선포인 0원** → 수익적립금에 모인 잔액 안에서만 배당 (재무 리스크 0%)

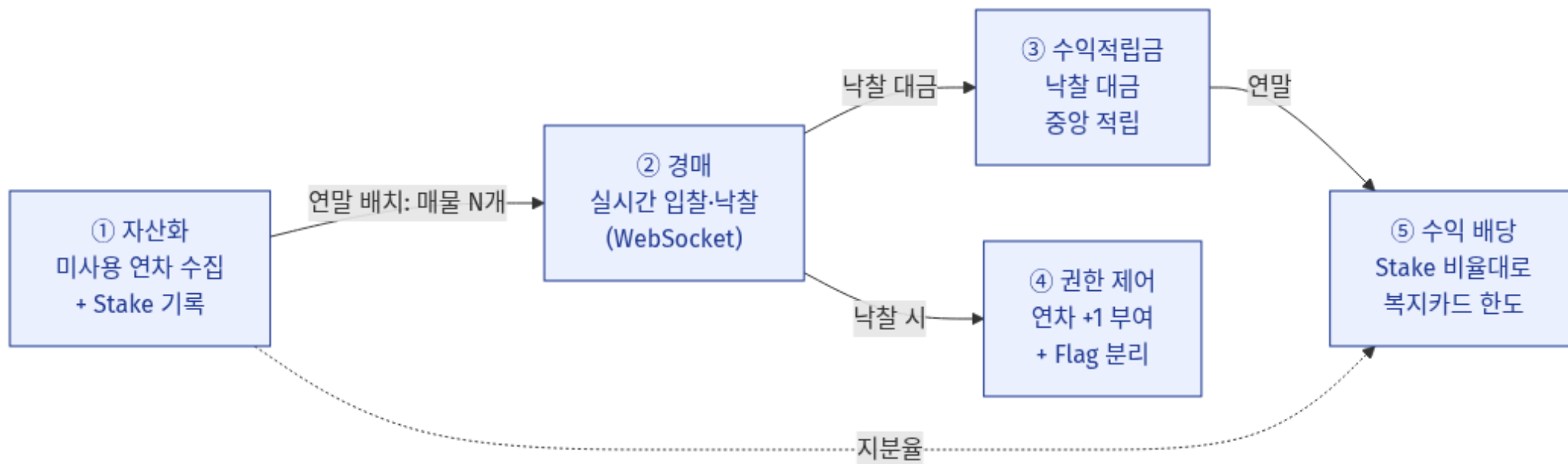
① 사용자(액터)와 3-Way Win

사용자 = 행위 기반 역할 (근속연수 아님): 직원(base) ← 판매자 · 구매자 · 관리자
외부 액터: 스케줄러(12/31 배치) · HR/ezpass 시스템

참여자	얻는 가치
회사/HR	예산 0원, 재무 리스크 0% — 비정규 연차는 수당 정산서 제외
판매자(고연차)	소멸될 0원 연차 → 연말 배당금(복지 한도)
구매자(저연차)	포인트로 부족한 연차 확보, 원하는 시기 휴식

동일 직원이 시점별로 판매자·구매자를 동시 수행 가능.

① 핵심 기능 — 5개 하위 시스템



② 요구사항 분석

기능 / 비기능 + 우선순위

② 기능 요구사항 (FR) + 우선순위

우선순위 = MoSCoW (M=Must / S=Should / C=Could)

Req ID	기능	우선순위
FR-2.1	실시간 입찰 (포인트 한도 내, 상위 입찰 WebSocket 알림)	M
FR-2.2	낙찰 및 수익적립금 적립 (포인트 차감 → 수익적립금에 누적)	M
FR-2.3	휴가 권한 부여 (낙찰 시 AUCTION 연차 +1, 단일 트랜잭션)	M
FR-3.1	차감 우선순위 제어 (AUCTION → EVENT → REGULAR)	M
FR-1.1	연말 정산·공용 풀 생성 (REGULAR 수집 → 매물 + Stake)	M
FR-4.1	연말 배당금 정산 (수익적립금 → Stake 비례 복지한도)	M
FR-5.2	잔액·거래내역 조회 (본인/관리자 전체)	S
FR-5.1	관리자 포인트 적립 (CREDIT_ADMIN , 사유 필수)	S
FR-4.2	유찰 재고 EVENT 수동 지급 + 연말 청산	C

② 비기능 요구사항 (NFR) — "사내 금융급 정합성"

ID	비기능 요구	실현 방법
NFR-1	동시성 — 마감 직전 동시 입찰 Race 0 건	SQLite write 락(lockAuction no-op UPDATE)으로 같은 경매 입찰 직렬화
NFR-2	재무 정합성·감사 추적 — 배당 오차 0원	거래 기록 Insert-Only + 수익적립금 등식 통화별 검증
(보안)	인증·권한	중앙 인증(ezpass) 위임 + JWT, RBAC(EMPLOYEE/ADMIN)
(실시간)	입찰가·타이머 즉시 반영	WebSocket 브로드캐스트

재무 정합성 불변식 (NFR-2 핵심):

$$\Sigma(\text{BID} + \text{WIN}) - \Sigma(\text{REFUND} + \text{DIVIDEND}) = \text{ESCROW.balance}$$

(통화별 · ESCROW.balance = 수익적립금 잔액)

→ 총 배당 = 수익적립금 총액. 1포인트도 초과 불가

② 절대 불변식 (DB-RULE) — 설계의 하중 부재

요구사항을 "깨지면 안 되는 규칙"으로 못 박은 4가지:

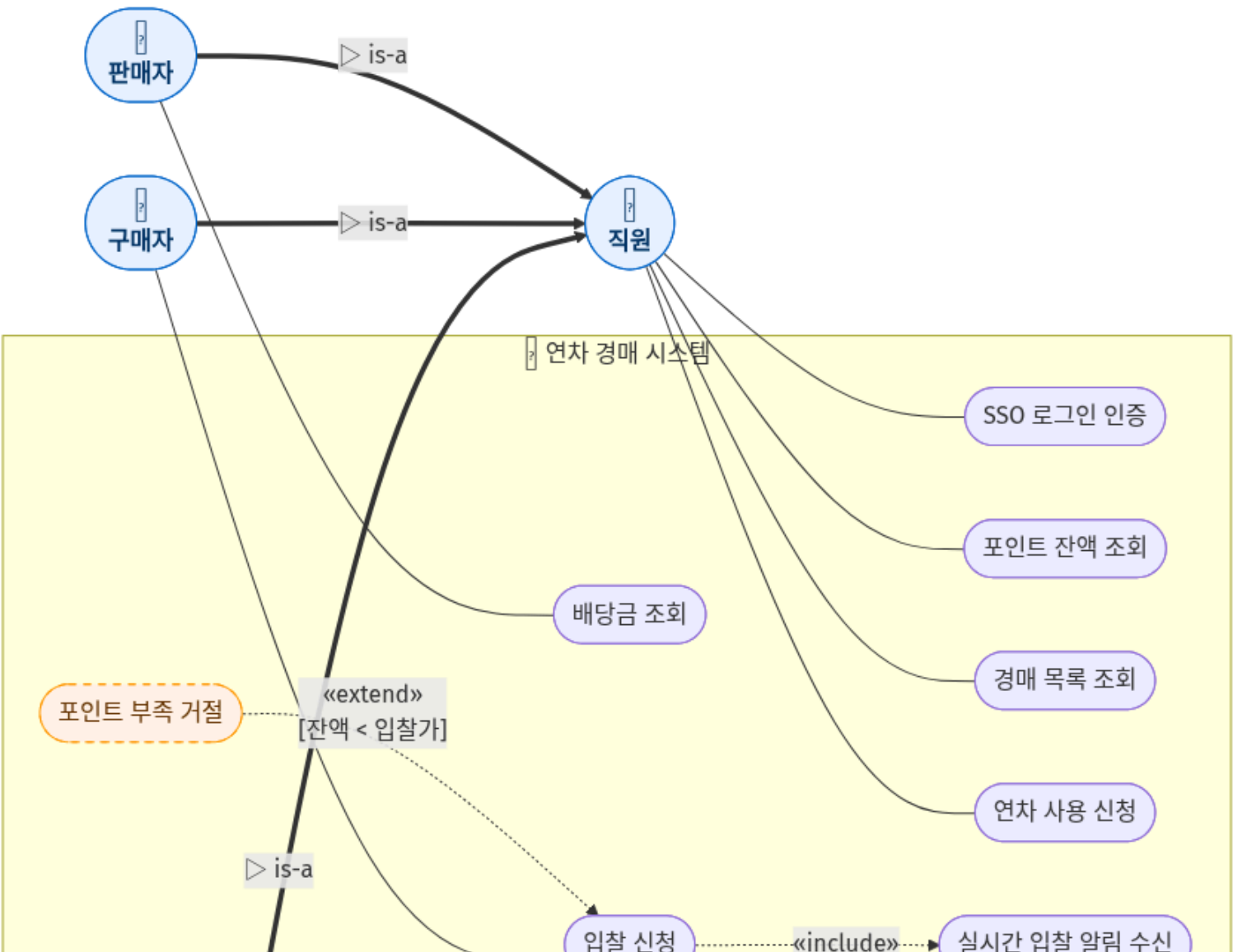
1. **거래 기록 불변 (Insert-Only)** — LEDGER_ENTRY 는 INSERT만, UPDATE/DELETE는 DB 트리거로 차단.
환불도 새 보정 INSERT.
2. **3-Flag 분리** — REGULAR (수당 O) / AUCTION (수당 X) / EVENT (수당 X) → 이중 보상(Double-Dipping) 사고 방지
3. **차감 우선순위 강제** — AUCTION → EVENT → REGULAR (사용자 선택 불가)
4. **수익적립금 상한** — 통화별 배당 합 = 수익적립금 잔액 (초과 금지)

이 시스템은 연차 게시판이 아니라 사내 복지 예산·HR 정산이 직결된 플랫폼 → 무결성이 1순위 요구사항.

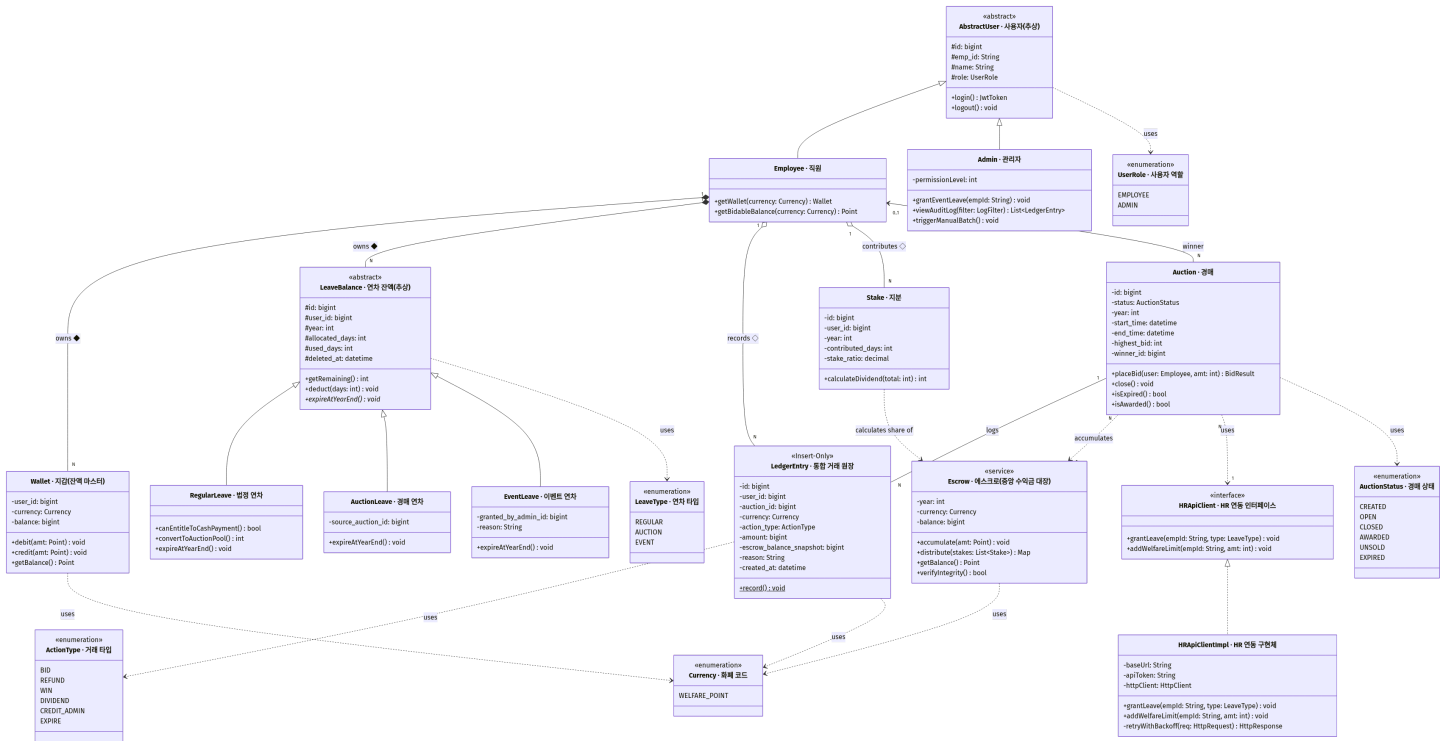
③ UML 모델링

기본 4종(유스케이스·클래스·순차·상태) + 심화 3종(활동·컴포넌트·객체)

③-1 유스케이스 다이어그램



③-2 클래스 다이어그램

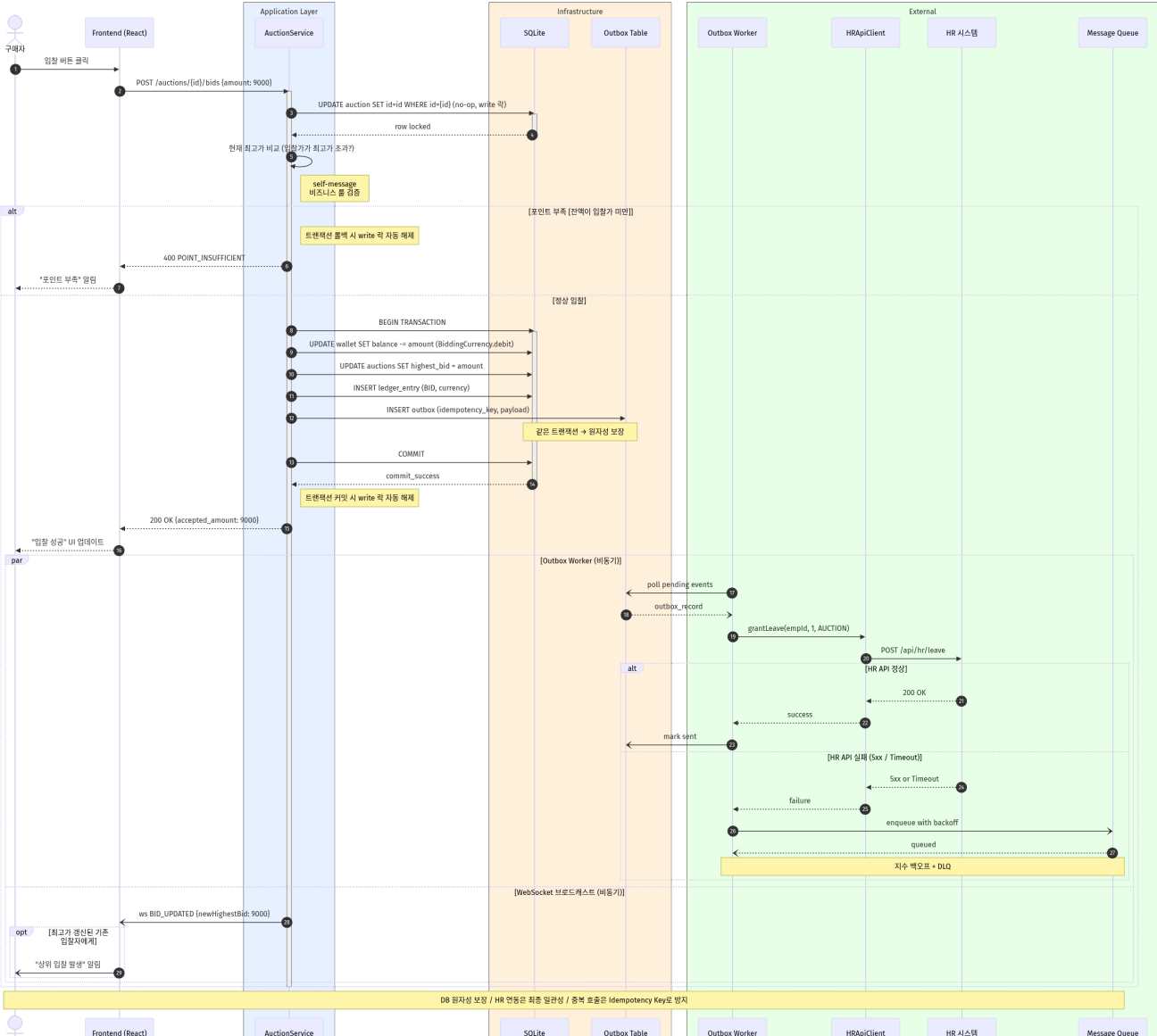


13 클래스 · 5 열거형 · 일반화 5 · 실체화 1 · 컴포지션/집합 · 가시성/다중도/스테레오타입 5종. LeaveBalance 3중 상속, HRApiClient 인터페이스

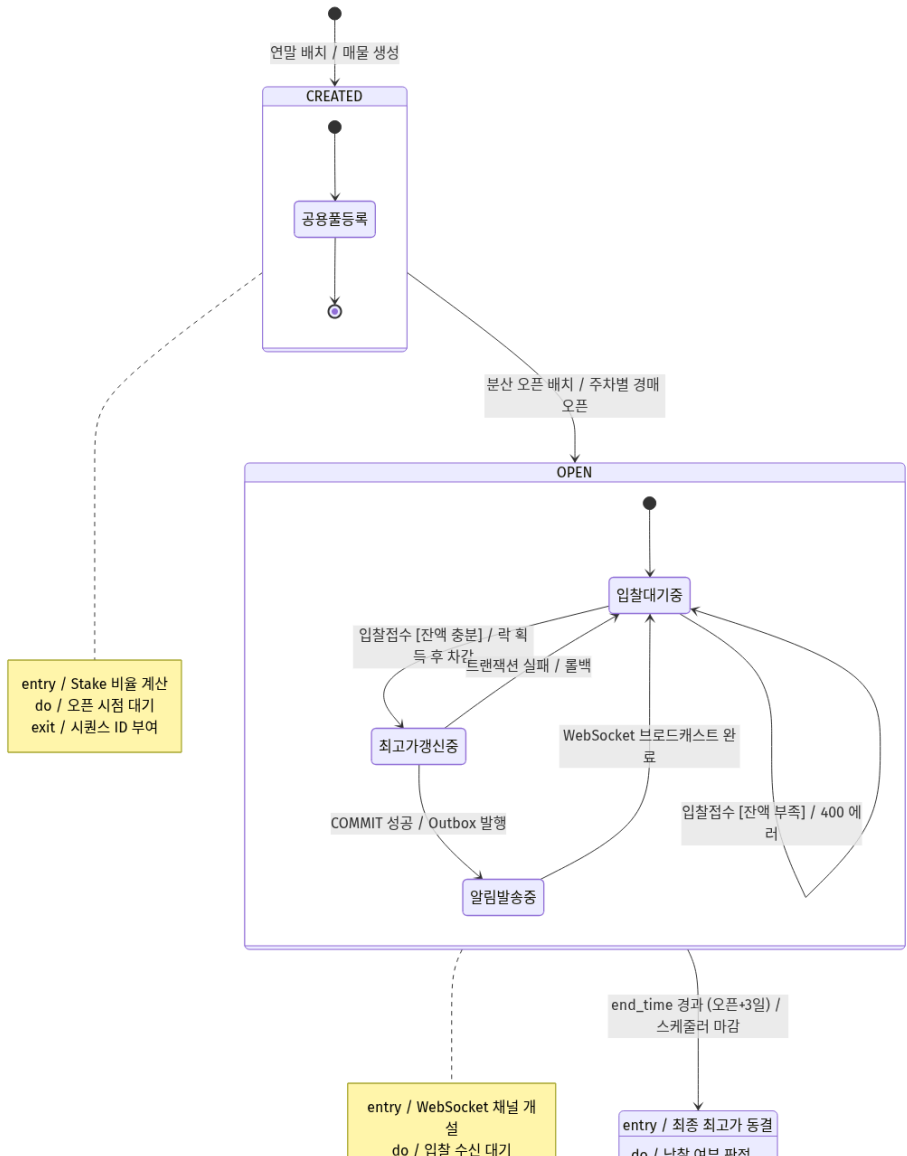


구현체, LedgerEntry «Insert-Only».

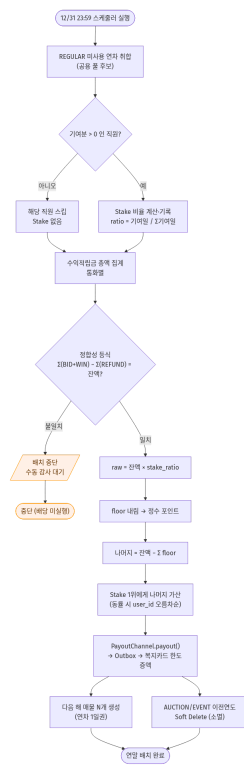
③-3 순차 다이어그램 — 입찰 → 낙찰 → 연차 부여



③-4 상태 다이어그램 — Auction 객체 생애주기



③-5 활동 다이어그램 — 연말 배치 (수익적립금 → 배당)



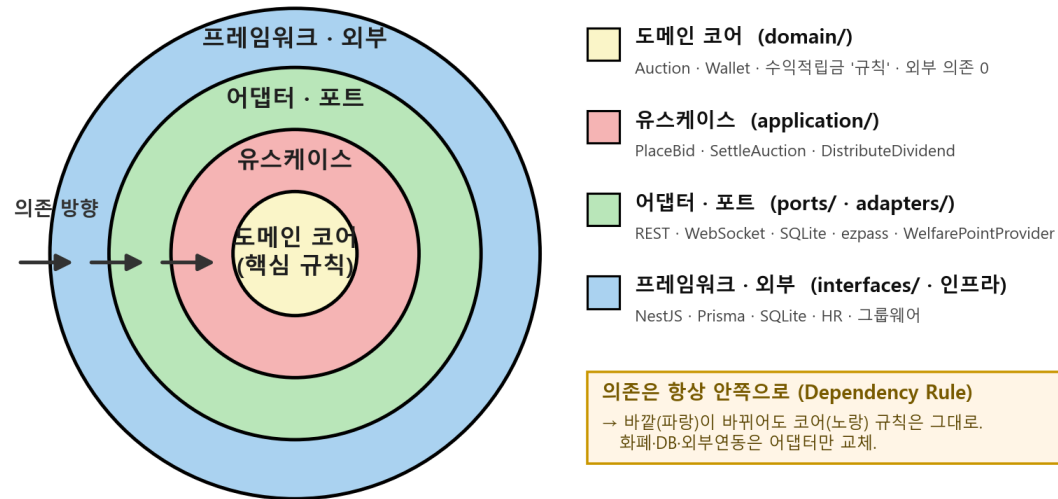
우리 프로젝트의 핵심 흐름. 1년간 쌓인 수익적립금 → **정합성 등식 검증** → 통과해야만 배당 산정(floor+나머지) → 매물 생성 ॥ 이전연도 소멸(병렬). 검증 실패 시 **배치 중단·수동 감사**. (FR-1.1·FR-4.1)

④ 소프트웨어 아키텍처

구조 선택 + 근거 + 블록 다이어그램

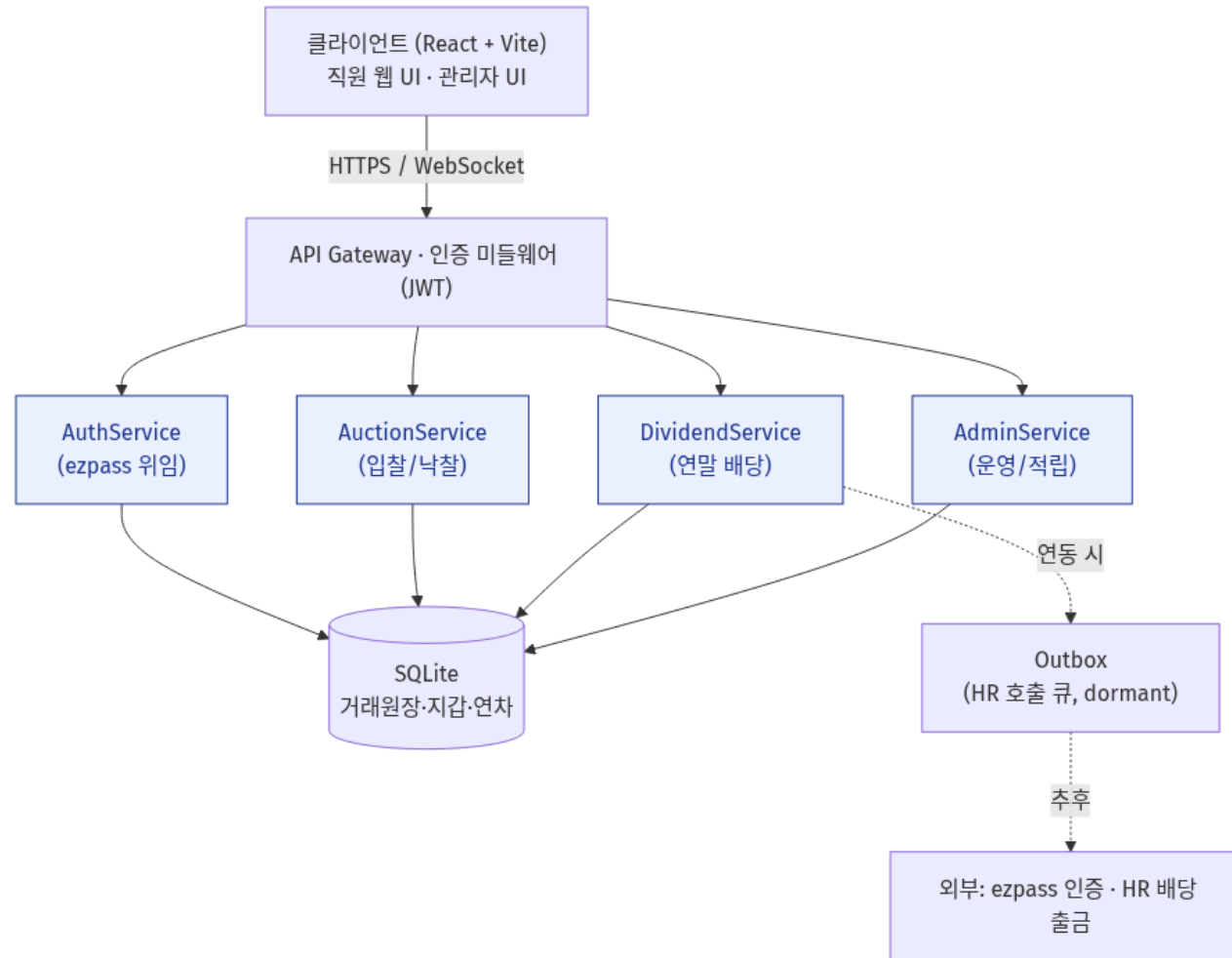
④ 아키텍처 — Hexagonal (Ports & Adapters)

헥사고널 / 클린 아키텍처 — 의존은 항상 안쪽으로



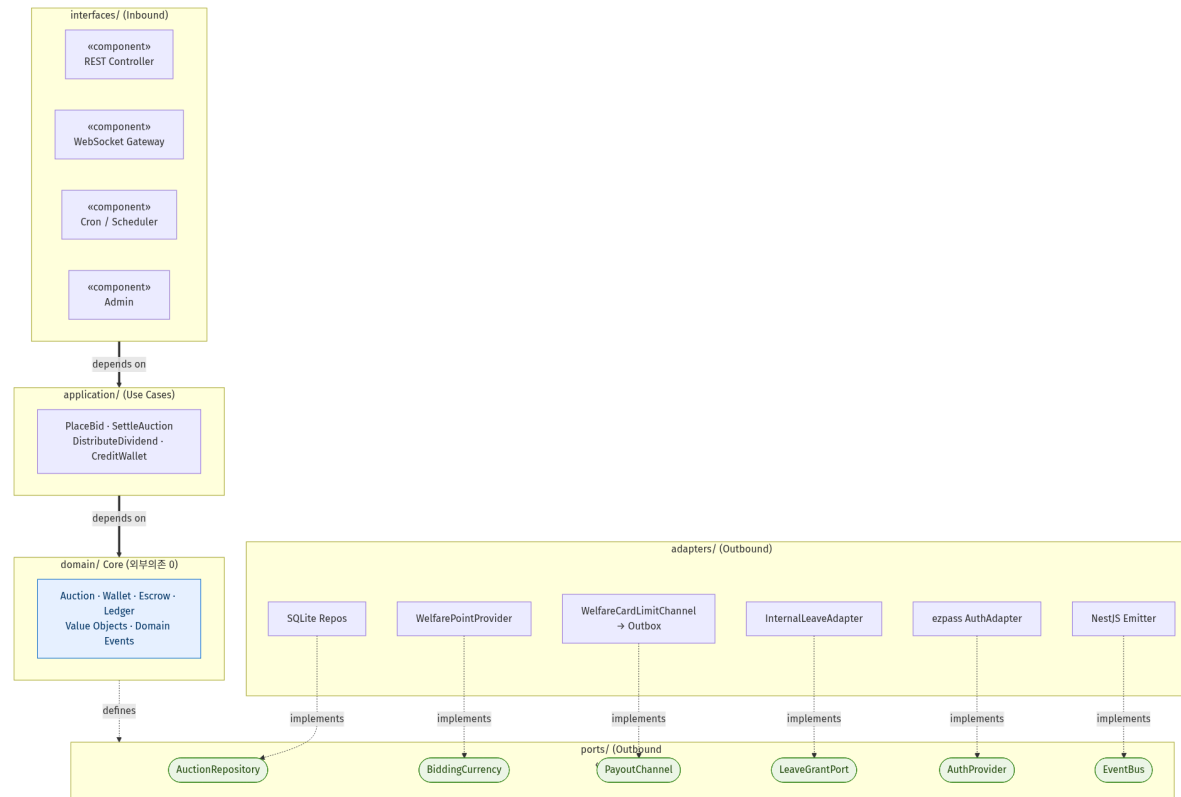
- 왜? 우리 시스템의 **핵심 도메인(규칙)**이 **외부 변화에 덜 흔들리게** 하려고 — DB·화폐·외부연동이 바뀌어도 **어댑터만 교체**, 코어는 그대로.
- **낙찰 정산 = 단일 트랜잭션**: 지갑 차감 + 적립 + 거래기록 + 연차부여 = All-or-Nothing.

④ 블록 다이어그램 — 논리 아키텍처 (MSA)



논리적으로 MSA로 분리, 물리적으로 단일 배포로 시작 — Hexagonal 경계가 미래 분리선.

④ 컴포넌트 다이어그램 — Ports & Adapters (정식 UML)



앞의 ASCII를 정식 UML로: «component» 계층 + «interface» 포트(BiddingCurrency·LeaveGrantPort·PayoutChannel·AuthProvider·EventBus)
 + 어댑터 **실체화(implements)**. 의존은 항상 안쪽으로. 같은 문서에 **5개 바운디드 컨텍스트 맵** 포함.

⑤ 적용 디자인 패턴

GoF — 어디에 · 왜

⑤ 적용 패턴 한눈에 (GoF 3종 + 보조)

패턴	분류	어디에	왜
Adapter	구조	<code>ports/</code>  <code>adapters/</code> (BiddingCurrency←WelfarePointProvider, AuthProvider←ezpass)	외부 자원 교체 가능, 도메인 무수정 (OCP)
Observer	행위	Domain Event — 입찰/낙찰 → 알림·메트릭·감사 구독	횡단 관심사 분리, Use Case가 부수효과를 모름
State	행위	<code>Auction</code> 상태 전이 (설계: 6상태)	<code>if(status==...)</code> 분기 제거, 상태별 규칙 캡슐화
보조: Strategy	행위	통화 Provider 교체 가능성	화폐 알고리즘 캡슐화
보조: Repository·UoW	-	영속성/트랜잭션 경계	도메인·DB 분리

→ 다음 2장에서 Adapter·Observer(구현 완료)와 State(설계 + 실용적 트레이드오프)를 상세히.

⑤-1 Adapter & Observer (코드에 실재)

Adapter (구조 패턴) — 헥사고날의 심장

- BiddingCurrency (포트) ← WelfarePointProvider (어댑터, wallet 테이블)
- LeaveGrantPort ← InternalLeaveAdapter (← 미래 GroupwareLeaveAdapter)
- AuthProvider ← ezpass 중앙 인증 어댑터
- 왜: 화폐·휴가·인증 등 외부 자원을 갈아끼워도 **도메인 코어 무수정**

Observer (행위 패턴) — 횡단 관심사 분리

- place-bid / settle-auction 유스케이스가 BidPlacedEvent · AuctionWonEvent **발행**
- 구독자: WebSocket 브로드캐스트 · 메트릭 · 감사 로그
- 왜: 새 부수효과를 추가할 때 Use Case를 **수정하지 않음** (개방-폐쇄)
- 구현: NestJS EventEmitter (@nestjs/event-emitter)

⑤-2 State 패턴 — 설계와 정직한 트레이드오프

설계: Auction 의 6상태를 상태 객체로 —

OpenState / ClosedState / AwardedState / UnsoldState / ExpiredState . 도메인 메서드에
if(status===...) 금지.

- **왜:** 상태별 허용 동작(입찰은 OPEN에서만)을 객체에 캡슐화 → 분기 폭증 방지

구현 현실 (scope-cuts CUT-3): 4상태 enum + 메서드 내부 guard 절로 실용적 단순화

- 근거: 2인 팀 · 단일 SQLite · 상태별 메서드 집합이 크게 갈라지지 않음
- **단, 외부 규칙은 동일:** 어댑터·Use Case는 .status 로 분기하지 **않고** placeBid()/settle() 호출 후
도메인 예외를 처리 → State 패턴의 의도는 보존

패턴을 "왜 쓰는가"뿐 아니라 "왜 이 범위에선 단순화했는가"까지 별도 설계 문서(scope-cut)로 남긴
것 — 적용의 타당성을 근거로 입증.

⑥ 역할 분담 + 14주차 구현 계획

⑥ 현재 구현 현황 (13주차 시점)

설계에 그치지 않고 **헥사고날 골격 + 핵심 경로**까지 구현 착수 완료.

영역	상태	근거
헥사고날 구조	✓	domain / application / ports / adapters / interfaces 분리
도메인 코어	✓	Auction · Wallet · Ledger · Value Object(Point/UserId/AuctionId) + 단위테스트
경매 Use Case	✓	place-bid · settle-auction · create-auction 등 6종
인증	✓	ezpass 중앙 인증 위임 통합·검증 완료
DB	✓	SQLite 전환 (외부 서버 의존 제거, Prisma)
프론트엔드	●	13개 화면(로그인·대시보드·경매·입찰·배당·관리자) 골격
연말 배치·배당	●	14주차 핵심 작업

⑥ 역할 분담 (옵션 A — 도메인 분할)

2인 팀 → "주담당 + 부담당(크로스 리뷰)" 체계.

담당	주담당 도메인	핵심 산출물
김기철	Auction / 입찰·낙찰 · 동시성 · 배치 · 인프라	입찰 API(write 락), 낙찰 정산, 12/31 배치, CI/CD
오지석	Auth · Leave · Dividend · Admin · 프론트 주도	차감 우선순위, Stake·배당 계산, 관리자 API, UI

- 의사결정: 기술 선택·주요 설계 결정은 **양자 합의**, 마이너 디테일은 담당 재량(PR 공유)
- 커뮤니케이션: GitHub PR(기술 논의 우선) · Issues(할 일) · 주간 스탠드업 30분
- 백업: 부재 시 부담당이 임시 주담당

⑥ 14주차 구현 계획 (마일스톤)

마일스톤	목표	완료 기준
M-A 경매 핵심 마감	입찰·낙찰 E2E	동시 입찰 부하 테스트(k6)서 입찰가 정합 100%
M-B 연차·차감	LeaveGrantPort ·차감 우선순위	AUCTION→EVENT→REGULAR 통합 테스트 통과
M-C 연말 배치·배당	12/31 풀 수집 + Stake + 배당	시드 데이터로 배당 합 = 수익적립금 검증(NFR-2)
M-D 관리자·실시간	유찰 EVENT 지급 · WebSocket	관리자 적립·유찰 지급 동작, 실시간 최고가 반영
M-E 통합·시연	E2E + 데모 시나리오	14주차 시연 스크립트대로 무중단 시연

리스크: ① 동시성(database is locked)→조기 부하테스트 ② 일학습병행 일정→가용시간 사전 공유 ③
프론트-백 스펙 불일치→OpenAPI 기준.

마무리 — 핵심 메시지

1. **합법성 × 재무 안전성**을 동시에 푼 *B2E Escrow & Dividend* 설계
2. 요구사항을 **불변식(DB-RULE)·NFR** 등식으로 못 박아 "정확성"을 검증 가능하게
3. **UML 4종**이 하나의 도메인 모델에서 일관 파생
4. **Hexagonal + GoF(Adapter·Observer·State)** 로 도메인 격리
5. 설계에 그치지 않고 **핵심 경로 구현 착수** → 14주차 시연으로 직결

감사합니다. 질문 환영합니다.

상세 설계 문서: [docs/](#) (SRS · UML · 설계 결정 22건 · ERD · OpenAPI)

부록 (Appendix)

질의응답 대비 백업 슬라이드

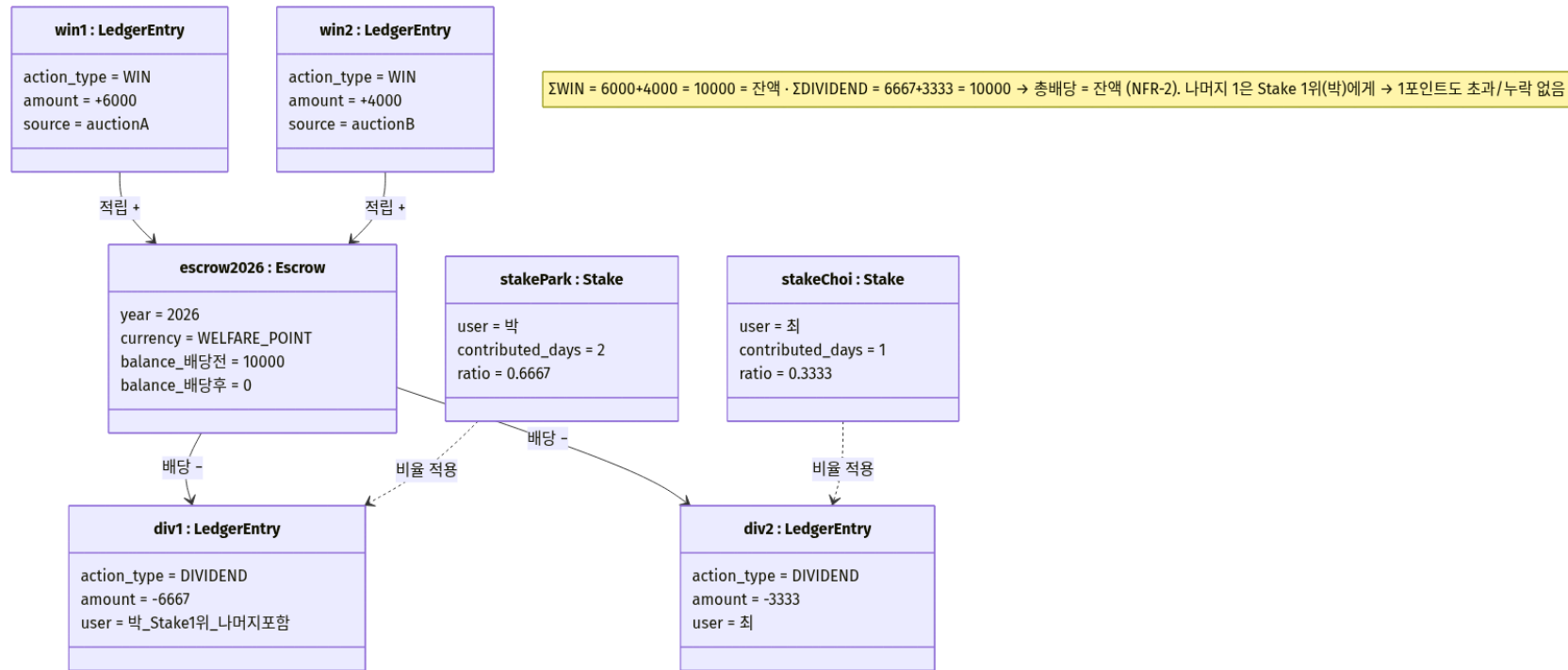
부록 A — 수익적립금 정합성, 어떻게 0원 오차?

배당 계산 (지분 비례 + 나머지 1위 몰아주기) — 아래 `escrow_balance` = 수익적립금 잔액

```
1. raw      = escrow_balance × stake_ratio(user)
2. floor    = [raw]                (정수 포인트 내림)
3. remainder= escrow_balance - Σ floor (나누어떨어지지 않는 잔여)
4. 최종배당 = floor + (Stake 1위면 remainder, 아니면 0)
```

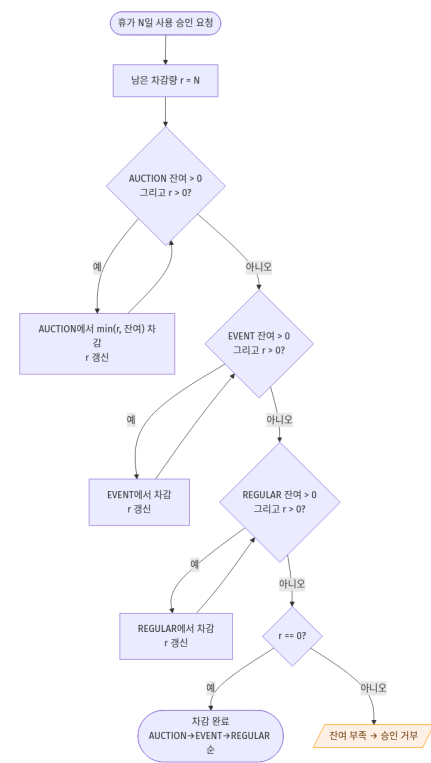
- 불변식: `Σ 최종배당 = escrow_balance` → **1포인트도 초과/미달 없음**
- Stake 동률: `user_id` 오름차순 등 결정적 타이브레이크
- 통화별 분리 계산 (DB-RULE-4)

부록 — 심화 UML ①: 객체 다이어그램 (정합성 스냅샷)



경매 A·B 낙찰 $\rightarrow$ 수익적립금 **10000**. 박(지분 2/3)·최(1/3) 배당: floor 6666/3333, 나머지 1은 Stake 1위(박) $\rightarrow$ 6667/3333. Σ 배당 = 10000 = 잔액 $\rightarrow$ NFR-2 등식이 숫자로 성립.

부록 — 심화 UML ②: 활동 다이어그램 (차감 우선순위)



사용자 선택 불가. **AUCTION → EVENT → REGULAR 강제 차감**. 소멸·수당제외 연차를 먼저 소진해 **REGULAR(수당·풀 전환 대상)**를 보존. 총 잔여 부족 시 승인 거부.

부록 B — 패자 환불 · 입찰 취소

- **밀리는 즉시 환불**: 상위 입찰 접수 시 직전 최고가 입찰자에게 같은 트랜잭션 내 `REFUND` INSERT + wallet 환급
- 최종 낙찰자만 차감 유지 → 마감 시 `WIN` 확정, 수익적립금 귀속
- **입찰 취소 불가** (GOV-3): 한번 입찰하면 되돌릴 수 없음 (가격 신뢰성)
- 관리자도 입찰 참여 가능(GOV-1) — 단, COI 대비 `LEDGER_ENTRY` 별도 플래그로 감사 분리

부록 C — 기술 스택 & 외부 자원 처리

영역	선택
백엔드	NestJS (TypeScript, Node 20)
ORM / DB	Prisma / SQLite (파일 기반, 외부 서버 0)
프론트	React + Vite
실시간	WebSocket (Socket.IO)
동시성	SQLite write 락 (<code>lockAuction</code>)
이벤트	NestJS EventEmitter (Observer)

3대 외부 자원(화폐·휴가·HR) 모두 동일 패턴: 포트 뒤로 추상화 → 내부 어댑터 기본 → 실 연동은 추후 어댑터 교체.

부록 D — 엣지 케이스 (발취)

상황	결정
연중 입사자	기여 불가 → <code>contributed_days=0</code> → Stake 없음
퇴사자	정책상 별도 처리 (edge-cases 카탈로그)
유찰 매물	수익적립금 적립 0 → 배당 자원 자연 감소 + 관리자 EVENT 수동 지급
자기 기여분 되사기	허용(행위 기반 역할) — <code>REGULAR→AUCTION</code> 전환은 대체로 자기 손해
HR API 5xx/Timeout (미래)	Outbox 재시도 + 지수 백오프 + DLQ (먹등 키)